AI Stock Research Dashboard

Engineering Deep Dive & Interview Study Guide

A full-stack financial research app: live market data, a multi-model LLM router (OpenAI / Claude), PostgreSQL caching, containers, automated tests and CI, and infrastructure-as-code for both Kubernetes and AWS.

Written to be understood by a curious 2nd-year CS student — so you can own every layer in an interview.

Ofir Eren
Portfolio project

What's inside

This guide is built around one promise: after reading it, you should be able to point at any box in your architecture and explain what it is, why you chose it, where it lives in the code, and what an interviewer might ask about it. Read it once end-to-end, then use the Interview boxes and the rapid-fire section as a cheat sheet before the interview.

- Part 1 - The project in plain English (the pitch).
- Part 2 - The request lifecycle: what happens when a user types a ticker.
- Part 3 - Technology deep dives (one per box in the diagram).
- Part 4 - The six bugs we fixed (shows engineering maturity).
- Part 5 - Cross-cutting concepts every interviewer probes.
- Part 6 - Trade-offs and what you'd do next (how to sound senior).
- Part 7 - Rapid-fire interview Q&A + a glossary.

> Remember: The single most impressive thing about this project is not any one technology - it is that it is a complete, honest slice of how real software ships: code, tests, CI, containers, and infrastructure. Lead with that.

Part 1 - The project in plain English

What it does

You type a stock ticker (say NVDA). The app fetches the last month of prices, works out how the stock performed versus its sector's benchmark ETF (its 'alpha'), pulls recent news headlines, and asks a large language model to write a short analyst-style note: a sentiment score, bull and bear cases, and how the company looks against its sector. The result is shown on a dashboard with a price chart, the numbers, the news, and the AI write-up.

Why this is a strong portfolio project

Most student projects are a single script or a single web app. This one is a small but complete system. It has a clear front end and back end talking over HTTP, a database used for a real purpose (caching), a pluggable AI layer, and it is packaged and deployable the way real teams ship software. Crucially, it also shows process: version control with clean commits, automated tests, a CI pipeline, and infrastructure described as code.

The one-sentence pitch

"A full-stack financial research app that pairs live market data with a multi-model LLM router, backs it with a PostgreSQL cache, and ships via Docker, GitHub Actions, Terraform (AWS ECS Fargate + RDS) and Kubernetes."

The technology map

Layer	Technology	Why it's here
Frontend	Streamlit	Turns Python into a web UI - chart, metrics, news, AI text.
Backend	FastAPI + Uvicorn	Typed HTTP API; one endpoint orchestrates the whole flow.
Market data	yfinance + pandas	Free Yahoo Finance data; pandas does the maths.
AI	OpenAI + Anthropic	A router picks the provider; falls back to a mock.
Cache	PostgreSQL + SQLAlchemy	Avoids re-fetching and re-paying the LLM per ticker.
Packaging	Docker + Compose	One command runs DB + backend + frontend together.
Quality	pytest + ruff + Actions	Tests and lint run automatically on every push.
Orchestration	Kubernetes (kind)	Run the containers in a real cluster, locally.
Cloud	AWS via Terraform	ECR, ECS Fargate, RDS, Secrets Manager, ALB - as code.

Part 2 - The request lifecycle

This is the single best thing to be able to recite in an interview. It connects every technology into one story. Here is exactly what happens when a user clicks 'Generate Comprehensive Report' for the ticker NVDA.

1. The browser shows the Streamlit dashboard. Streamlit's Python server makes an HTTP GET to the backend: GET /api/stock/NVDA (with a 30-60s timeout, because the call can be slow).
2. FastAPI receives the request in read_stock(). It upper-cases the ticker and first asks the cache: get_cached_analysis('NVDA').
3. The cache (PostgreSQL via SQLAlchemy) is checked. If there is a row newer than the TTL (1 hour), it is returned immediately with "cached": true - no data fetch, no LLM call, no cost. This is the fast path.
4. On a cache miss, finance_service.get_stock_info() runs: it reads the price from yfinance fast_info, the descriptive fields (sector, name, summary) from yfinance .info, one month of price history, and recent news. pandas turns the price series into a one-month return.
5. It maps the stock's sector to a benchmark ETF (e.g. Technology -> XLK), fetches the ETF's one-month return the same way, so the UI can show alpha = stock return - ETF return.
6. ai_service.get_ai_analysis() builds a prompt from that data and routes it: with an Anthropic key it calls Claude; otherwise OpenAI; with no key it returns a deterministic mock. The provider SDK call carries a timeout so a hung provider can't block forever.
7. The combined result (finance data + AI text) is written back to PostgreSQL via save_analysis() - best-effort: if the DB is down it just skips caching.
8. FastAPI returns JSON: { finance_data, ai_analysis, cached: false }.
9. Streamlit receives the JSON and renders: the price chart, the four metric tiles (price, sector, ETF, alpha), the news links, and the AI write-up.

Remember: If they ask 'walk me through what happens on a request', this list is your answer. Notice how it naturally name-drops every layer: HTTP, FastAPI, cache, yfinance, pandas, the AI router, and JSON.

Part 3 - Technology deep dives

Each section follows the same shape: a one-line definition, why it is in this project, where it lives in the code, how our code actually uses it, and the questions an interviewer is likely to ask.

3.1 FastAPI - the backend and API

In one line

FastAPI is a modern Python web framework for building HTTP APIs, known for speed, type hints, and automatic interactive docs.

Why it's here

Something has to receive 'give me data for NVDA' and orchestrate the work. FastAPI lets you write that as a normal typed Python function and turns it into a real HTTP endpoint, with request validation and auto-generated documentation at /docs for free.

Where in the project

app/main.py - the app object, the CORS configuration, the lifespan startup hook, and the /api/stock/{ticker} route.

How our code uses it

```
@app.get("/api/stock/{ticker}")
def read_stock(ticker: str):
    ticker = ticker.upper()
    try:
        cached = get_cached_analysis(ticker)
        if cached is not None:
            return cached
        finance_data = get_stock_info(ticker)
        ai_insight = get_ai_analysis(finance_data, news_titles)
        save_analysis(ticker, finance_data, ai_insight)
        return {"finance_data": finance_data, "ai_analysis": ai_insight, "cached": False}
    except ValueError:
        raise HTTPException(status_code=404, detail="Ticker not found ...")
    except Exception:
        raise HTTPException(status_code=500, detail="An internal error occurred ...")
```

Two details worth understanding. First, the path parameter {ticker} is automatically typed as a str and injected as the function argument. Second, we split error handling: a ValueError (e.g. an invalid ticker with no data) becomes a 404 'not found', while anything unexpected becomes a 500 - and in both cases the full error is logged on the server while the client only sees a generic message (never leak internals to users).

We also use a 'lifespan' function: a startup hook that initialises the database connection once when the server boots, instead of racing to do it on the first request.

Interview angle

Q: What is FastAPI and why use it over Flask?
A: It's an async-capable Python API framework with type-hint-based validation and auto-generated OpenAPI docs. Versus Flask you get request validation and docs without extra libraries, and first-class async support.

Q: What's the difference between a 404 and a 500, and why did you separate them?
A: 404 means the resource doesn't exist (a bad ticker) - the client's fault and expected; 500 means an unexpected server error. Separating them gives honest status codes and lets clients react correctly.

Q: Why log the real error but return a generic message?
A: Security and UX: detailed errors can leak stack traces, file paths or secrets, and mean nothing to a user. We keep the detail in server logs for debugging.

3.2 Streamlit - the frontend

In one line

Streamlit turns a plain Python script into an interactive web app - no HTML, CSS or JavaScript required.

Why it's here

It lets a Python-focused project have a genuinely nice UI with charts and inputs in a few lines, so the demo looks like a product rather than a terminal.

Where in the project

dashboard.py - the whole front end.

How our code uses it

```
ticker = st.text_input("Enter Stock Ticker:", "NVDA").upper()
if st.button("Generate Comprehensive Report"):
    response = requests.get(f"{BACKEND_URL}/api/stock/{ticker}", timeout=30)
    if response.status_code == 200:
        data = response.json()
        st.line_chart(data["finance_data"]["history"])
        st.markdown(data["ai_analysis"])
```

Important architectural point: the frontend and backend are separate programs. Streamlit does not import the backend code - it talks to it over HTTP, exactly like a real browser would. That separation is why we can containerise and scale them independently.

Interview angle

Q: Why is the frontend a separate service instead of one big app?
A: Separation of concerns and independent deployment/scaling. The UI can change or restart without touching the API, and each gets its own container and resources.

Q: Where does the BACKEND_URL come from?
A: An environment variable. Locally it points at 127.0.0.1; in Docker Compose it's http://backend:8000; in Kubernetes/AWS it's the internal service address. Same code, different config.

3.3 yfinance + pandas - the market data

In one line

yfinance is a Python library that pulls free market data from Yahoo Finance; pandas is the standard library for tabular data and numerical work in Python.

Why it's here

We need real prices, history and news without paying for a data vendor. yfinance gives that; pandas makes computing a return from a price series trivial.

Where in the project

app/services/finance_service.py

How our code uses it

```
stock = yf.Ticker(ticker)
try:
    info = stock.info or {}          # slow, descriptive fields (sector, name, summary)
except Exception:
    info = {}                        # tolerate a flaky .info instead of crashing
try:
    fast_price = stock.fast_info.last_price   # fast, reliable price
except Exception:
    fast_price = None
hist = stock.history(period="1mo")
prices = hist["Close"].tolist()
stock_return = ((prices[-1] / prices[0]) - 1) * 100   # one-month % return
```

Two resilience lessons live here. (1) yfinance's .info is the slowest and flakiest call, so we wrap it in try/except and fall back to fast_info for the price - a transient Yahoo hiccup degrades the output rather than returning a 500. (2) We compute the return defensively, guarding against an empty list or a zero first price (avoiding a divide-by-zero).

The sector-to-ETF comparison is the analytical heart: we map the stock's sector to a benchmark ETF (Technology -> XLK, Energy -> XLE, ...), fetch the ETF's return the same way, and the difference is 'alpha' - did the stock beat its peer group.

Interview angle

Q: What is the riskiest external dependency here and how did you handle it?

A: yfinance's .info call - it throttles and breaks without notice. I take the price from the faster fast_info endpoint and wrap .info so a failure degrades gracefully.

Q: What is 'alpha' in your app?

A: The stock's one-month return minus its sector ETF's return - a simple measure of whether it outperformed its peer group rather than just rode the market.

3.4 The multi-model LLM router - OpenAI and Claude

In one line

A small abstraction that sends the same prompt to whichever AI provider is configured - Anthropic's Claude or OpenAI - and falls back to a deterministic mock when no key is set.

Why it's here

Tying the whole app to one vendor is fragile and a weak interview story. A router shows you understand abstraction: one prompt-builder, several interchangeable backends, chosen by configuration rather than code

edits. The mock means the app runs end-to-end with zero keys.

Where in the project

app/services/ai_service.py

How our code uses it

```
def _resolve_provider():
    requested = os.getenv("AI_PROVIDER", "auto").lower()
    has_claude = bool(os.getenv("ANTHROPIC_API_KEY"))
    has_openai = bool(os.getenv("OPENAI_API_KEY"))
    if requested == "claude": return "claude" if has_claude else "mock"
    if requested == "openai": return "openai" if has_openai else "mock"
    if has_claude: return "claude"   # auto: prefer Claude, then OpenAI, then mock
    if has_openai: return "openai"
    return "mock"
```

Each provider is one small adapter (_analyze_with_claude, _analyze_with_openai) behind a shared _build_prompt(). Both SDK clients are created with a timeout so a hung provider can't tie up a worker. We default to cost-efficient models (gpt-4o-mini, claude-haiku-4-5) and let you override via env (OPENAI_MODEL, CLAUDE_MODEL) - a deliberate cost/quality decision.

Interview angle

Q: How would you add a third provider, say Gemini?
A: Write one function _analyze_with_gemini(prompt) using its SDK, add a branch in the router and a key check. Nothing else changes - that's the point of the abstraction.

Q: Why default to the cheaper models?
A: Cost. The endpoint calls an LLM per cache-miss; the small models are plenty for a short structured note, and anyone can bump the model via env for deeper analysis.

Q: What happens with no API keys at all?
A: The router returns 'mock' and the app serves a canned analysis, so it still runs end-to-end for demos and tests without credentials.

3.5 PostgreSQL + SQLAlchemy - the caching layer

In one line

PostgreSQL is a powerful open-source relational database; SQLAlchemy is the Python toolkit that lets us talk to it with Python objects instead of raw SQL.

Why it's here

Fetching data and calling an LLM is slow and, for the LLM, costs money. Caching each ticker's result for an hour means repeat requests are instant and free. It also gives the project a real, justified database - not a database bolted on for show.

Where in the project

app/db.py (engine + the StockCache table) and app/services/cache_service.py (read/write).

How our code uses it

```
class StockCache(Base):
    __tablename__ = "stock_cache"
    ticker = Column(String, primary_key=True)
    data = Column(JSON, nullable=False)        # the finance payload
```

```
    ai_analysis = Column(Text, nullable=False)
    updated_at = Column(DateTime, default=utcnow, nullable=False)

# read: return the row only if it is younger than the TTL
if row and (utcnow() - row.updated_at) &lt; CACHE_TTL:
    return {"finance_data": row.data, "ai_analysis": row.ai_analysis, "cached": True}
```

The standout design choice is graceful degradation: the database is initialised lazily and, if Postgres is unreachable, we log one warning and disable caching instead of crashing. The app keeps serving live results. Reads and writes are wrapped so a cache failure never breaks the request path - the cache is an optimisation, not a dependency.

Remember: This 'the cache is an optimisation, not a hard dependency' line is gold in an interview - it shows you think about failure modes, not just the happy path.

Interview angle

Q: Why a database and not just an in-memory dict?
A: An in-memory cache dies on restart and isn't shared across multiple backend instances. A database persists and is shared, which matters once you scale to more than one container.

Q: What is an ORM and what does SQLAlchemy give you?
A: Object-Relational Mapping: you work with Python classes/objects and it generates the SQL. It also parametrises queries, which prevents SQL injection.

Q: How is the cache invalidated?
A: By time (TTL). A row older than one hour is treated as a miss and refreshed. Simple and good enough for research data that doesn't need to be real-time.

3.6 Docker + Docker Compose - packaging

In one line

Docker packages an app with everything it needs into a portable image that runs the same anywhere; Docker Compose runs several such containers together as one system.

Why it's here

'Works on my machine' is the classic problem. A container freezes the OS libraries, Python version and dependencies so the app behaves identically on your laptop, in CI, and in the cloud. Compose then wires the database, backend and frontend into one command.

Where in the project

Dockerfile.backend, Dockerfile.frontend, docker-compose.yml

How our code uses it

```
# docker-compose.yml (shortened)
services:
  db:        # PostgreSQL, with a pg_isready healthcheck
  backend:   # build Dockerfile.backend; DATABASE_URL points at "db"
    depends_on:
      db: { condition: service_healthy }   # wait for the DB to be ready
  frontend:  # BACKEND_URL points at "http://backend:8000"
    depends_on:
      backend: { condition: service_healthy }
```

Two details that read as 'this person has used Docker for real'. (1) Health checks: each container reports whether it's actually ready, and dependents wait for that - so the backend doesn't start before the database can accept connections. (2) The health check uses Python's urllib, not curl, because the slim Python base image doesn't ship curl - a small but real gotcha.

We also split dependencies: the frontend image installs only Streamlit + requests, not the backend's heavy data/AI libraries, keeping it lean.

Interview angle

Q: What's the difference between an image and a container?
A: An image is the immutable blueprint (built once); a container is a running instance of that image. Same image, many containers.

Q: What does Docker Compose add over plain Docker?
A: It defines and runs a multi-container system - networking, env vars, build, startup order and health gating - from one declarative file and one command.

Q: Why health checks?
A: So orchestration knows a container is actually ready (not just started) before sending it traffic or starting things that depend on it.

3.7 pytest + ruff + GitHub Actions - quality and CI

In one line

pytest runs automated tests, ruff is a fast linter that flags code issues, and GitHub Actions runs both automatically every time you push code.

Why it's here

Tests prove the logic works and keep it working as you change things. A linter keeps the code consistent. CI ('Continuous Integration') runs them for you on every push, so a broken change is caught in minutes - this is the single clearest signal of professional habits.

Where in the project

tests/ (19 tests), pyproject.toml (pytest + ruff config), .github/workflows/ci.yml

How our code uses it

```
# A test mocks yfinance with REAL pandas DataFrames + side_effect,
# so there is no network and the __getitem__ trap is avoided:
def test_etf_return_guarded_against_empty_history():
    stock = _make_ticker({...}, [100, 110])
    etf   = _make_ticker({}, [])          # empty ETF history
    with patch("...yf.Ticker", side_effect=[stock, etf]):
        result = get_stock_info("XOM")
    assert result["etf_return_1mo"] == 0.0   # guarded, no crash
```

The tests are designed to need no network and no credentials: yfinance, the LLM SDKs and the database are all mocked (the cache tests use an in-memory SQLite database). That's why CI can run them in seconds. The CI workflow has four jobs: lint+tests, a Docker build of both images, terraform validate, and kubeconform on the Kubernetes manifests - so even the infrastructure code is checked automatically.

Interview angle

Q: What is the difference between unit tests and integration tests?
A: Unit tests check one piece in isolation (with dependencies mocked); integration tests check pieces working together. Mine are mostly unit tests plus a real-DB cache round-trip using in-memory SQLite.

Q: What does your CI pipeline actually do?
A: On every push/PR it lints with ruff, runs pytest, builds both Docker images, validates the Terraform, and schema-checks the Kubernetes manifests. Green CI means all of that passed.

Q: CI vs CD?
A: CI (Continuous Integration) automatically builds and tests every change. CD (Continuous Deployment) would also auto-deploy it. I have CI; deployment is a manual Terraform step.

3.8 Kubernetes (kind) - orchestration

In one line

Kubernetes is the industry-standard system for running and managing containers across machines; 'kind' runs a real Kubernetes cluster locally inside Docker for development.

Why it's here

Compose is great on one machine; Kubernetes is what teams use to run containers in production at scale, with self-healing, scaling and rolling updates. Writing real manifests - and running them locally on kind - is strong evidence you understand orchestration, not just buzzwords.

Where in the project

k8s/ - manifests for the namespace, Postgres, backend, frontend, an autoscaler, and a kind config that maps the dashboard to localhost:8501.

Core objects you should be able to name

- Deployment - declares 'I want N copies of this container'; Kubernetes keeps that true (restarts crashed pods, replaces them on a node failure).
- Service - a stable internal address/load-balancer in front of a Deployment's pods, so 'backend' always resolves even as pods come and go.
- ConfigMap / Secret - non-secret and secret configuration injected as environment variables.
- PersistentVolumeClaim (PVC) - durable storage for Postgres so data survives a pod restart.
- Liveness / readiness probes - health checks Kubernetes uses to restart or route around pods.
- HorizontalPodAutoscaler (HPA) - automatically adds/removes backend pods based on CPU.

A neat detail: the backend builds its DATABASE_URL from the database Secret using Kubernetes' $(VAR) interpolation, so the password is never written in plaintext in a ConfigMap. AI keys are referenced as 'optional', so the app still starts (in mock mode) if they're absent.

Interview angle

Q: Deployment vs Pod vs Service?
A: A Pod is one running instance (one or more containers). A Deployment manages a set of identical Pods and keeps them running. A Service is the stable network endpoint that load-balances across them.

Q: Why does Postgres use a PVC?
A: Pods are ephemeral - their filesystem is lost on restart. A PVC is durable storage so the database keeps its data across restarts.

Q: Liveness vs readiness probe?
A: Liveness: 'is it alive?' - if not, restart it. Readiness: 'can it take traffic yet?' - if not, don't route to it. A pod can be alive but not ready (still warming up).

3.9 AWS + Terraform - cloud infrastructure as code

In one line

AWS is Amazon's cloud; Terraform lets you describe cloud infrastructure in text files and create or destroy it with one command, instead of clicking around a web console.

Why it's here

'Infrastructure as Code' (IaC) means your servers, database and networking are version-controlled, reviewable, reproducible, and destroyable (so you stop paying). This is exactly how modern teams manage cloud, and it is a standout line on a student CV.

Where in the project

terraform/ - one .tf file per concern (network, ecr, rds, secrets, iam, ecs, alb, outputs).

The AWS pieces and what each one is

- VPC + subnets - your own private network in AWS. We use two public subnets and no NAT gateway (tasks get public IPs) to keep cost down.
- ECR (Elastic Container Registry) - a private Docker image registry; you push the backend and frontend images here.
- ECS Fargate - runs your containers without you managing any servers ('serverless containers'). You hand it an image and a CPU/memory size; AWS runs it.
- Cloud Map (service discovery) - gives the backend an internal DNS name (backend.stock-ai.local:8000) so the frontend can find it.
- RDS - a managed PostgreSQL database (AWS handles backups, patching). It's private - only the app's security group can reach it.
- Secrets Manager - stores the DATABASE_URL and AI keys; ECS injects them into the container at runtime, so secrets are never baked into the image.
- ALB (Application Load Balancer) - the public entry point that routes internet traffic to the frontend.
- IAM roles - tightly scoped permissions; e.g. the task's execution role may read exactly the three secrets and nothing else.

How our code uses it

```
# terraform/ecs.tf - secrets injected from Secrets Manager (not in the image)
secrets = [
  { name = "DATABASE_URL",     valueFrom = aws_secretsmanager_secret.database_url.arn },
  { name = "OPENAI_API_KEY",   valueFrom = aws_secretsmanager_secret.openai.arn },
  { name = "ANTHROPIC_API_KEY",valueFrom = aws_secretsmanager_secret.anthropic.arn },
```

```
]
```

Remember: Be honest in interviews: say you wrote and validated the Terraform/Kubernetes (CI runs terraform validate and kubeconform), ran the stack locally on kind, and the AWS config is deploy-ready. Don't claim you ran a paid production EKS/ECS cluster unless you actually did.

Interview angle

Q: What is 'infrastructure as code' and why is it better than the console?
A: Defining infra in version-controlled text. It's reproducible, reviewable in pull requests, and you can destroy it to stop costs. Clicking in a console is none of those.

Q: What is Fargate and how is it different from EC2?
A: Fargate runs containers without you managing the underlying servers - you specify CPU/memory and AWS handles the host. With EC2 you manage the virtual machines yourself.

Q: How are secrets handled in the cloud?
A: They live in AWS Secrets Manager. ECS injects them as environment variables at runtime via the task's IAM execution role, so they're never in the image or the code.

Q: How does the frontend find the backend on AWS?
A: Cloud Map service discovery gives the backend a private DNS name; the frontend's BACKEND_URL points at backend.stock-ai.local:8000 inside the VPC.

Part 4 - The six bugs we fixed

Before adding infrastructure, the codebase had real bugs. Being able to talk about finding and fixing them shows maturity - debugging is most of real engineering.

1. API path mismatch. The frontend called /api/stock/... but the backend route was /stock/... - so every request 404'd. Fix: align both on /api/stock/{ticker}.

2. Silent empty fields. Descriptive fields were read from yfinance fast_info, which doesn't contain sector/name/summary, so they came out blank. Fix: read those from stock.info.

3. Divide-by-zero on the ETF return. The stock return was guarded against an empty list / zero price, but the ETF return wasn't - a crash waiting to happen. Fix: same guard.

4. Broken default summary. An empty summary became just '...'. Fix: a real default text and truncate only when the summary is genuinely long.

5. Leaking raw errors. Exceptions were returned to the client verbatim and always as 500. Fix: log the full error server-side, return a generic message, and use 404 for not-found vs 500 for unexpected.

6. CORS misconfiguration. A wildcard origin ('*') combined with credentials is invalid and insecure. Fix: pin a specific allowed origin (configurable per environment).

Remember: The meta-point to make: 'I verified each bug against the running code before fixing it - I didn't just trust the list.' That sentence signals rigour.

Part 5 - Cross-cutting concepts interviewers probe

CORS (Cross-Origin Resource Sharing)

Browsers block a page on one origin from calling an API on another origin unless the API opts in. Our backend allows a specific frontend origin. Note: the frontend-to-backend call is server-side (Streamlit's server calls the API), so CORS mainly matters for safety and correctness rather than being on the critical path.

Environment-based configuration (the 12-factor idea)

Nothing environment-specific is hard-coded: the database URL, backend URL, allowed origin, AI provider and model, and TTL all come from environment variables. The same image runs locally, in Compose, in Kubernetes and on AWS - only the config changes. Secrets stay out of the code and out of git.

Graceful degradation

The system has three deliberate fallbacks: no DB -> caching disabled, app still serves; no AI key -> deterministic mock; flaky yfinance .info -> price from fast_info and 'Unknown' fields. The app prefers to give a degraded answer over crashing.

Secrets management

Locally: a .env file that is gitignored. In Kubernetes: Secret objects. On AWS: Secrets Manager injected at runtime. The principle is identical everywhere - secrets are never in the code or the image.

Stateless services vs stateful data

The backend and frontend are stateless - any copy can handle any request, which is what makes horizontal scaling (more pods/tasks) possible. All state lives in PostgreSQL. This split is the foundation of scalable systems.

Health checks and service discovery

Health checks let the platform know when a container is ready (Compose gating, Kubernetes probes, the ALB target check). Service discovery (Kubernetes Services, AWS Cloud Map) gives components stable names to find each other as instances come and go.

Part 6 - Trade-offs and what you'd do next

Naming your own limitations is what makes you sound senior, not junior. These are honest, deliberate trade-offs - have an answer ready for each.

Schema management: create_all, not migrations

The single cache table is created on boot. That's fine for one table but can't evolve columns safely. In production I'd use Alembic migrations. (Stated in the README too.)

Sync endpoints, not async

The API uses synchronous handlers that FastAPI runs in a threadpool. Simple and fine at this scale; the LLM/yfinance calls carry timeouts so a slow upstream can't pin a worker forever. A high-throughput version would go fully async.

No auth or rate limiting on a paid endpoint

Anyone who knows the URL could trigger LLM calls. Before exposing it publicly I'd add an API key or rate limiting; for a demo it's acceptable.

RDS in public subnets

To avoid a paid NAT gateway, RDS sits in public subnets but is not publicly accessible - only the app's security group can reach it. Private subnets would be stricter at extra cost.

What I'd build next

- A real CD step: GitHub Actions builds and pushes images to ECR and rolls the ECS service.
- Caching keyed by content, and a background refresh so users never wait on a cold call.
- Auth + rate limiting, and observability (metrics/tracing) on the backend.
- Alembic migrations and a managed Terraform state backend (S3 + DynamoDB lock).

Part 7 - Rapid-fire interview Q&A

Q: Give me the 30-second overview.

A: A full-stack stock research app: a Streamlit UI calls a FastAPI backend, which fetches market data with yfinance, compares the stock to its sector ETF, asks an LLM (Claude or OpenAI, chosen by config) for an analyst note, and caches the result in PostgreSQL. It's containerised with Docker, tested in CI with GitHub Actions, and deployable to Kubernetes or AWS via Terraform.

Q: Why both a frontend and a backend?

A: Separation of concerns and independent scaling. The API can be reused or scaled separately from the UI, and each is its own container.

Q: Where's the slow part and how did you handle it?

A: yfinance and the LLM. I cache results in Postgres for an hour, put timeouts on the LLM calls, and take the price from yfinance's fast endpoint.

Q: How does it scale?

A: The services are stateless, so you run more containers/pods behind a load balancer; state is in Postgres. Kubernetes has an HPA that scales the backend on CPU.

Q: What happens if the database is down?

A: Caching silently turns off and the app keeps serving live results - the cache is an optimisation, not a hard dependency.

Q: How do you keep secrets safe?

A: Env vars from a gitignored .env locally; Kubernetes Secrets; AWS Secrets Manager injected at runtime. Never in code or images.

Q: What does green CI guarantee?

A: Lint passed, all tests passed, both Docker images build, the Terraform validates, and the Kubernetes manifests are schema-valid.

Q: What was the hardest part?

A: Pick a real one: e.g. wiring service-to-service networking consistently across Compose, Kubernetes and AWS Cloud Map, or making the tests fully offline by mocking yfinance/LLM/DB.

Glossary

API - A contract for one program to call another. Here, HTTP endpoints the frontend calls.

Endpoint / route - A specific URL the API responds to, e.g. GET /api/stock/{ticker}.

ORM - Object-Relational Mapping - work with DB rows as Python objects (SQLAlchemy).

TTL - Time To Live - how long a cached item is considered fresh (1 hour here).

Container / image - Image = blueprint; container = a running instance of it (Docker).

Orchestration - Running/managing many containers across machines (Kubernetes).

IaC - Infrastructure as Code - infra defined in text and applied with a tool (Terraform).

Fargate - Run containers on AWS without managing servers (serverless containers).

VPC - Your private virtual network inside AWS.

Load balancer - Distributes incoming traffic across multiple instances (ALB).

Service discovery - Letting components find each other by stable name (Cloud Map / K8s Service).

CI / CD - Continuous Integration (auto build+test) / Continuous Deployment (auto deploy).

Stateless - A service that keeps no local state, so any instance can serve any request.

Graceful degradation - Giving a reduced-but-working result instead of failing completely.

How to tell the story

In an interview, don't list technologies - tell the lifecycle (Part 2) and let the technologies appear naturally. Then be ready to go one level deeper on whichever box they point at, using Part 3. Finish weak points with the honest trade-offs from Part 6. The combination - a working system plus the humility to name its limits - is exactly what interviewers want from a strong junior engineer.

Built with curiosity. Now go own the room.